

Naive Bayes Classifier

Lab 1: Naive Bayes Classifier on the Iris Dataset

Objective: Classify iris species based on sepal and petal measurements.

1. Import Libraries and Dataset:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target
```

2. Split the Dataset:

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)
```

3. Train the Model:

```
model = GaussianNB()
model.fit(X_train, y_train)
```

4. Make Predictions and Evaluate:

```
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print(f"Accuracy: {accuracy}")
```

The `accuracy_score` is a metric used to evaluate the performance of a classification model. It is the ratio of the number of correct predictions to the total number of predictions made. This metric gives an overall indication of how often the classifier is correct.

Formula

Accuracy = Number of correct predictions / Total number of predictions

Now compare the accuracy of the models using other evaluation technique line CONFUSION MATRIX:

- A confusion matrix provides detailed insight into the performance of a classification model by showing the true positives, true negatives, false positives, and false negatives. This helps in understanding the types of errors the model makes.

5. Calculate and Print the Confusion Matrix and Accuracy:

```
from sklearn.metrics import confusion_matrix

# Calculate and Print the Confusion Matrix and Accuracy
confusion_matrix_iris = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(confusion_matrix_iris)

# Calculate accuracy from confusion matrix
cm_accuracy = confusion_matrix_iris.trace() / confusion_matrix_iris.sum()
print(f"Confusion Matrix Accuracy: {cm_accuracy}")
```

Lab 2: Naive Bayes Classifier on the SMS Spam Collection Dataset

Objective: Classify SMS messages as spam or ham (non-spam).

1. Import Libraries and Dataset:

```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score

# Load dataset
url = "https://raw.githubusercontent.com/justmarkham/pycon-2016-tutorial/master/data/sms.tsv"
df = pd.read_csv(url, sep='\t', header=None, names=['label', 'message'])
```

2. Preprocess Data:

```
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(df['message'])
y = df['label']
```

3. Split the Dataset:

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)
```

4. Train the Model:

```
model = MultinomialNB()
model.fit(X_train, y_train)
```

5. Make Predictions and Evaluate:

```
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy}")
```

Explanation:

1. **Load the Dataset:**
 - The dataset is loaded from a URL into a pandas DataFrame.
2. **Preprocess Data:**
 - `CountVectorizer` is used to convert the SMS text data into a matrix of token counts.
3. **Split the Dataset:**
 - The dataset is split into training and test sets using `train_test_split`.
4. **Train the Model:**
 - A `MultinomialNB` (Naive Bayes classifier for multinomial models) is trained on the training data.
5. **Make Predictions:**
 - The model makes predictions on the test set.
6. **Evaluate the Model:**
 - Accuracy is calculated using `accuracy_score`.

Sources

1. datacamp.com - Naive Bayes Classifier Tutorial: with Python Scikit-learn
2. analyticsvidhya.com - Naive Bayes Classifier: Definition, Applications and ...
3. kdnuggets.com - Naïve Bayes Algorithm: Everything You Need to Know
4. machinelearningmastery.com - Naive Bayes Classifier From Scratch in Python
5. machinelearningplus.com - How Naive Bayes Algorithm Works?

Task for Lab 2

Evaluate and print the CONFUSION MATRIX and Accuracy and details CLASSIFICATION REPORT.

Solution:

6. Calculate and Print the Confusion Matrix and Accuracy:

```
from sklearn.metrics import confusion_matrix, classification_report

# Calculate and Print the Confusion Matrix and Accuracy
conf_matrix = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(conf_matrix)

# Calculate accuracy from confusion matrix
cm_accuracy = conf_matrix.trace() / conf_matrix.sum()
print(f"Confusion Matrix Accuracy: {cm_accuracy}")
```

7. Print the Classification Report:

```
# Print Classification Report
class_report = classification_report(y_test, y_pred)
print("Classification Report:")
print(class_report)
```

Precision

Precision is the ratio of correctly predicted positive observations to the total predicted positives. It answers the question: Of all instances that were predicted positive, how many were actually positive?

$$\text{Precision} = \frac{TP}{TP+FP}$$

Where TP is True Positives and FP is False Positives.

Recall

Recall, also known as Sensitivity or True Positive Rate, is the ratio of correctly predicted positive observations to all observations in the actual class. It answers the question: Of all instances that were actually positive, how many were predicted correctly?

$$\text{Recall} = \frac{TP}{TP+FN}$$

Where TP is True Positives and FN is False Negatives.

F1-score

The F1-score is the harmonic mean of Precision and Recall. It is a better measure than accuracy for imbalanced classes, as it considers both false positives and false negatives.

$$\text{F1-score} = 2 \times \left(\frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \right)$$

Support

Support refers to the number of actual occurrences of each class in the dataset. It is the number of true instances for each label.

$$\text{Support} = TP + FN$$

Macro Average

Macro average calculates the metric independently for each class and then takes the average (mean) of those metrics. This treats all classes equally, regardless of their support (the number of true instances for each class). It is particularly useful when you want to treat all classes equally, without giving more importance to any class based on its frequency.

Example:

If you have precision scores of 0.8, 0.7, and 0.9 for three classes, the macro average precision is:

$$\text{Macro Average Precision} = \frac{0.8+0.7+0.9}{3} = 0.8$$

Weighted Average

Weighted average calculates the metric for each class, and then finds their average, weighted by the number of true instances for each class (support). This gives more importance to the classes with more instances. It is useful when you want the evaluation to reflect the performance across all samples, giving more weight to classes with more instances.

Example:

If you have precision scores of 0.8, 0.7, and 0.9 for three classes with supports of 50, 30, and 20 respectively, the weighted average precision is:

$$\text{Weighted Average Precision} = \frac{(0.8 \times 50) + (0.7 \times 30) + (0.9 \times 20)}{50 + 30 + 20} = 0.79$$